

```
13. alternate_prices = prices[::2] # [142.56, 144.22, 145.03, 148.56]
14.
```

The slicing syntax `list[start:end:step]` is incredibly powerful for time series analysis. The `start` index is inclusive, the `end` index is exclusive, and the `step` allows you to skip elements.

*?? **Pro Tip:** When calculating technical indicators like moving averages or momentum oscillators, use slicing to create a rolling window of recent prices. For example, a 5-day simple moving average could be calculated with `recent_prices = all_prices[-5:]` followed by `average = sum(recent_prices) / len(recent_prices)`. This technique is used by professional quants at firms like AQR Capital Management and Two Sigma to efficiently implement indicator-based trading strategies without complex loops.*

Modifying Lists: Updating Your Trading Data

Unlike some data structures, lists are mutable—meaning they can be changed after creation. This is useful for updating trading data as new information arrives:

```
1. portfolio = ["AAPL", "MSFT", "GOOGL"]
2.
3. # Change an element
4. portfolio[1] = "AMZN" # Replace MSFT with AMZN
5.
6. # Check the updated portfolio
7. print(portfolio) # ["AAPL", "AMZN", "GOOGL"]
8.
9. You can also add elements to a list using several methods:
10. # Add a stock to the end of the portfolio
11. portfolio.append("NFLX") # ["AAPL", "AMZN", "GOOGL", "NFLX"]
12.
13. # Add multiple stocks at once
14. new_stocks = ["TSLA", "META"]
15. portfolio.extend(new_stocks) # ["AAPL", "AMZN", "GOOGL", "NFLX", "TSLA", "META"]
16.
17. # Insert a stock at a specific position
18. portfolio.insert(2, "IBM") # Insert IBM at index 2
19. # Result: ["AAPL", "AMZN", "IBM", "GOOGL", "NFLX", "TSLA", "META"]
```